Exercise #1

$$A = \begin{pmatrix} -b_1 & b_2 \\ b_3 & -b_1 \end{pmatrix}$$

$$A - \lambda I = 0$$

$$\begin{pmatrix} -b_1 & b_2 \\ b_3 & -b_1 \end{pmatrix} - \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix} =$$

$$\begin{pmatrix} -b_1 - \lambda & b_2 \\ b_3 & -b_1 - \lambda \end{pmatrix}$$

$$\det(A - \lambda I) = 0$$

$$(-b_1 - \lambda)^2 - (b_2 b_3) = 0$$

$$(b_1 + \lambda)^2 = b_2 b_3$$

$$\lambda = \pm\sqrt{b_2 b_3} - b_1$$

$$\lambda_1 = \sqrt{b_2 b_3} - b_1 \qquad \lambda_2 = -\sqrt{b_2 b_3} - b_1$$

<u>Case 1:</u> both λ's are neg.

Since $b_1, b_2, b_3 > 0 \Rightarrow \lambda_2$ always neg.

to get λ_1 neg.

$$\hookrightarrow \sqrt{b_2 b_3} - b_1 < 0$$

$$b_2 b_3 < b_1^2$$

<u>Case 2:</u> one λ pos., other λ neg.

λ_2 always neg., so $\lambda_1 > 0$

$$\sqrt{b_2 b_3} - b_1 > 0$$

$$b_2 b_3 > b_1^2$$

Daniel Chang

M150 Lab 2

Exercise 2

```
% copied and pasted the matrix from the question
af = [-1, -0.5, -0.5, -0.5, -0.5;
      -0.5,  -1, -0.5, -0.5, -0.5;
      -0.5, -0.5,  -1, -0.5, -0.5;
      -0.5, -0.5, -0.5,  -1, -0.5;
      -0.5, -0.5, -0.5, -0.5,  -1]
```

```
af = 5×5
   -1.0000   -0.5000   -0.5000   -0.5000   -0.5000
   -0.5000   -1.0000   -0.5000   -0.5000   -0.5000
   -0.5000   -0.5000   -1.0000   -0.5000   -0.5000
   -0.5000   -0.5000   -0.5000   -1.0000   -0.5000
   -0.5000   -0.5000   -0.5000   -0.5000   -1.0000
```

```
% given vector
v=[1;2;0;3;1]
```

```
v = 5×1
     1
     2
     0
     3
     1
```

```
% applying it 6 times
u1=af*v/norm(af*v);
u2=af*u1/norm(af*u1);
u3=af*u2/norm(af*u2);
u4=af*u3/norm(af*u3);
u5=af*u4/norm(af*u4);
u6=af*u5/norm(af*u5)
```

```
u6 = 5×1
    0.4472
    0.4472
    0.4472
    0.4472
    0.4472
```

```
% lambda estimate after the 6 iterations
lambda_est = u6' * af * u6
```

```
lambda_est =
-3.0000
```

```
% vector of eigenvalues from the given matrix
true_eigs = eig(af)
```

```
true_eigs = 5×1
   -3.0000
   -0.5000
   -0.5000
   -0.5000
   -0.5000
```

```
% index corresponding to the largest magnitude eigenvalue in the vector
[~, idx] = max(abs(true_eigs));
% setting largest absolute eigenvalue in the vector to true_largest
true_largest = true_eigs(idx)
```

```
true_largest =
-3.0000
```

```
% finding the error difference between the eigenvalue estimate and the true
% eigenvalue
err = abs(lambda_est - true_largest)
```

```
err =
6.0940e-10
```

The lambda estimate done from the algorithm is extremely close to 0. The difference being between the lambda estimate and the true eigenvalue 6.094 x 10^-10.

Exercise 3

```
% getting a random 5 x 5 matrix and setting it equal to A
A = rand(5)
```

```
A = 5×5
    0.2810    0.5181    0.6761    0.2548    0.7805
    0.4401    0.9436    0.2891    0.2240    0.6753
    0.5271    0.6377    0.6718    0.6678    0.0067
    0.4574    0.9577    0.6951    0.8444    0.6022
    0.8754    0.2407    0.0680    0.3445    0.3868
```

```
% adding A and its tranpose together to symmetrize the matrix guaranteeing
% real eigenvalues
A = A + A'
```

```
A = 5×5
    0.5620    0.9581    1.2033    0.7122    1.6559
    0.9581    1.8872    0.9268    1.1817    0.9160
    1.2033    0.9268    1.3436    1.3630    0.0747
    0.7122    1.1817    1.3630    1.6888    0.9466
    1.6559    0.9160    0.0747    0.9466    0.7735
```

```
% 200 iterations
n_iterations = 200;

% Run 1: first initial vector
v1 = [1; 0; 1; 0; 1];
% creating an empty vector of zeros that is length corresponding to # of
% iterations
ll1 = zeros(n_iterations, 1);

% running the algorithm for the set # of iterations for this initial vector
u = v1;
for i = 1:n_iterations
    u = A*u / norm(A*u);
    ll1(i) = u' * A * u;
end
```

```
% Run 2: different initial vector
v2 = [3; -1; 2; 4; -2];
ll2 = zeros(n_iterations, 1);

% running the algorithm for the set # of iterations for this initial vector

u = v2;
for i = 1:n_iterations
    u = A*u / norm(A*u);
    ll2(i) = u' * A * u;
end

% true largest eigenvalue
true_eigs = eig(A);
[~, idx] = max(abs(true_eigs));
true_largest = true_eigs(idx);

fprintf('Run 1 estimate: %.6f\n', ll1(end));
```

```
Run 1 estimate: 5.296764
```

```
fprintf('Run 2 estimate: %.6f\n', ll2(end));
```

```
Run 2 estimate: 5.296764
```

```
fprintf('True largest:   %.6f\n', true_largest);
```

```
True largest:   5.296764
```

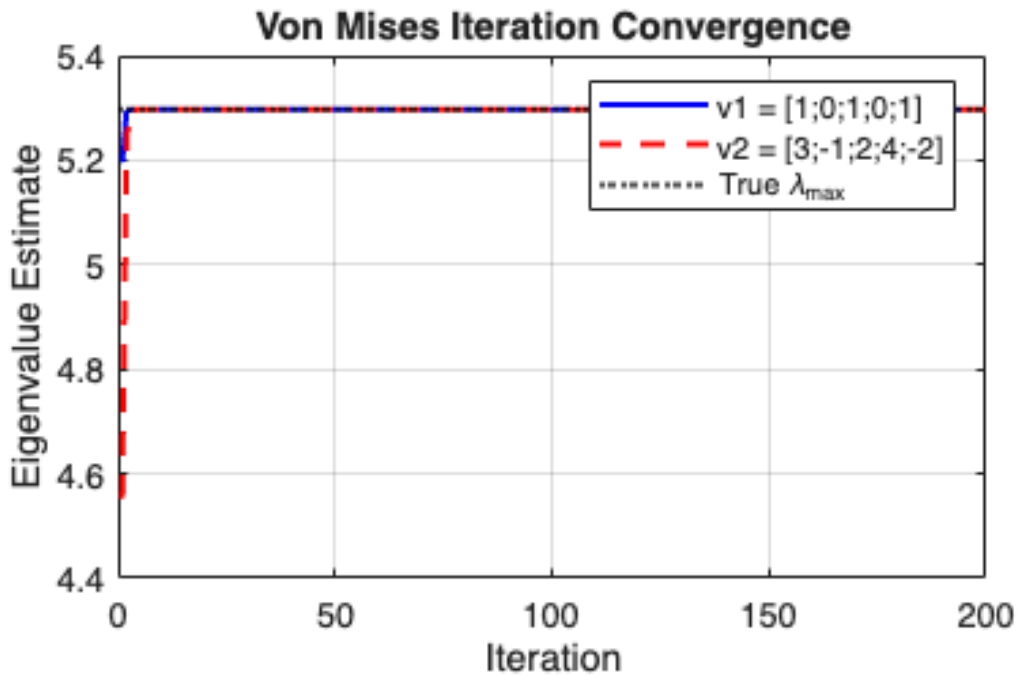
```
fprintf('Run 1 error: %.2e\n', abs(ll1(end) - true_largest));
```

```
Run 1 error: 1.78e-15
```

```
fprintf('Run 2 error: %.2e\n', abs(ll2(end) - true_largest));
```

```
Run 2 error: 1.78e-15
```

```
% plotting the first set and second set of iterations on the same plot
% along with the true eigenvalue as a horizontal black dotted line
figure;
% graph corresponding to the first set of iterations of v1 (blue dotted
% line)
plot(1:n_iterations, ll1, 'b-', 'LineWidth', 1.5); hold on;
% graph corresponding to the second set of iterations of v2 (red dotted
% line)
plot(1:n_iterations, ll2, 'r--', 'LineWidth', 1.5);
% graph corresponding to the true dominant eigenvalue from eig()
yline(true_largest, 'k:', 'LineWidth', 1.5);
% labeling the graph
xlabel('Iterations'); ylabel('Eigenvalue Estimate');
title('Von Mises Iteration Convergence');
legend('v1 = [1;0;1;0;1]', 'v2 = [3;-1;2;4;-2]', 'True \lambda_{max}');
grid on;
```



Do you get the same answer with a different initial vector?

Yes, you get the same answer with a different initial vector. Both v1 and v2 in my code converged to the same eigenvalue estimate of `5.296764. The eigenvalue estimate also matches true eigenvalue from the eig() function of 5.296764.`

Does rate of convergence depend on the initial guess?

Based on my plot, both vectors seem to converge immediately to the true dominant eigenvalue in the first couple of iterations. As a result, the rate of converge does not seem to be meaningfully affected by the initial vector of choice.

Why choose a symmetric matrix?

A + A' guarantees the matrix to be symmetric. This makes sure that all the eigenvalues from the matrix are real.

What happens if you use a non-symmetric matrix with a complex conjugate pair as the dominant eigenvalue?

The iteration would oscillate because the the conjugate pair contains two complex eigenvalues of the same magnitude (one positive , other negative). That being said, oscillations would occur due to the method bounce between both associated directions of the pair of eigenvalues.